

~ -zsh...memory\_and\_test\_shell.c~ nano signal\_handler.c~ -zsh~ appen

UW PICO 5.09File: combined\_complex.c

#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <fcntl.h>  
#include <sys/wait.h>  
  
#define MAX\_CMDS 10  
#define MAX\_ARGS 32  
#define BUF\_SIZE 1024  
  
// Helper function to trim spaces  
char \*trim\_space(char \*str) {  
 while (\*str == ' ' || \*str == '\t') str++;  
 if (\*str == 0) return str;  
 char \*end = str + strlen(str) - 1;  
 while (end > str && (\*end == ' ' || \*end == '\t')) end--;  
 end[1] = '\0';  
 return str;  
}  
  
// Applies input/output/error redirections to a single command stage  
void apply\_redirection(char \*cmd\_buf) {  
 char \*input\_file = NULL;  
 char \*output\_file = NULL;  
 int append\_out = 0;  
 int merge\_stderr = 0;  
 char \*stderr\_file = NULL;  
 int append\_err = 0;  
  
 // 1. Check for combined shorthand redirection (&> file)  
 char \*amp\_out\_pos = strstr(cmd\_buf, "&>");  
 if (amp\_out\_pos) {  
 \*amp\_out\_pos = '\\0';  
 output\_file = trim\_space(amp\_out\_pos + 2);  
 merge\_stderr = 1;  
 }  
  
 // 2. Check for stderr redirection to stdout (2>&1)  
 char \*merge\_pos = strstr(cmd\_buf, "2>&1");  
 if (merge\_pos) {  
 \*merge\_pos = '\\0';  
 merge\_stderr = 1;  
 }  
  
 // 3. Check for standard stderr redirection (2>> or 2>)  
 if (!merge\_stderr) {  
 char \*err\_app\_pos = strstr(cmd\_buf, "2>>");

^G Get Help  
^X Exit

^O WriteOut  
^J Justify

^R Read File  
^W Where is

^Y Prev Pg  
^V Next Pg

^K Cut Text  
^U UnCut Text

~ -zsh...memory\_and\_test\_shell.c~ nano signal\_handler.c~ -zsh~ appen

UW PICO 5.09File: combined\_complex.c

char \*output\_file = NULL;  
int append\_out = 0;  
int merge\_stderr = 0;  
char \*stderr\_file = NULL;  
int append\_err = 0;  
  
// 1. Check for combined shorthand redirection (&> file)  
char \*amp\_out\_pos = strstr(cmd\_buf, "&>");  
if (amp\_out\_pos) {  
 \*amp\_out\_pos = '\\0';  
 output\_file = trim\_space(amp\_out\_pos + 2);  
 merge\_stderr = 1;  
}  
  
// 2. Check for stderr redirection to stdout (2>&1)  
char \*merge\_pos = strstr(cmd\_buf, "2>&1");  
if (merge\_pos) {  
 \*merge\_pos = '\\0';  
 merge\_stderr = 1;  
}  
  
// 3. Check for standard stderr redirection (2>> or 2>)  
if (!merge\_stderr) {  
 char \*err\_app\_pos = strstr(cmd\_buf, "2>>");  
 char \*err\_tr\_pos = strstr(cmd\_buf, "2>");  
 if (err\_app\_pos) {  
 append\_err = 1;  
 \*err\_app\_pos = '\\0';  
 stderr\_file = trim\_space(err\_app\_pos + 3);  
 } else if (err\_tr\_pos) {  
 append\_err = 0;  
 \*err\_tr\_pos = '\\0';  
 stderr\_file = trim\_space(err\_tr\_pos + 2);  
 }  
}  
  
// 4. Check for stdout redirection (>> or >)  
if (!output\_file) {  
 char \*out\_app\_pos = strstr(cmd\_buf, ">>");  
 char \*out\_tr\_pos = strchr(cmd\_buf, '>');  
 if (out\_app\_pos) {  
 append\_out = 1;  
 \*out\_app\_pos = '\\0';  
 output\_file = trim\_space(out\_app\_pos + 2);  
 } else if (out\_tr\_pos) {  
 append\_out = 0;  
 \*out\_tr\_pos = '\\0';  
 output\_file = trim\_space(out\_tr\_pos + 1);  
 }  
}

^G Get Help  
^X Exit

^O WriteOut  
^J Justify

^R Read File  
^W Where is

^Y Prev Pg  
^V Next Pg

^K Cut Text  
^U UnCut Text

~ - -zsh...memory\_and\_test\_shell.c~ - - nano signal\_handler.c~ - -zsh~ - - appen

UW PICO 5.09File: combined\_complex.c

```
    }
}

// 5. Check for input redirection (<)
char *in_pos = strchr(cmd_buf, '<');
if (in_pos) {
    *in_pos = '\0';
    input_file = trim_space(in_pos + 1);
}

// Process Input Redirection
if (input_file && strlen(input_file) > 0) {
    int in_fd = open(input_file, O_RDONLY);
    if (in_fd < 0) {
        perror("Input file open error");
        exit(EXIT_FAILURE);
    }
    dup2(in_fd, STDIN_FILENO);
    close(in_fd);
}

// Process Output Redirection
if (output_file && strlen(output_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (append_out ? O_APPEND : O_TRUNC);
    int out_fd = open(output_file, flags, 0644);
    if (out_fd < 0) {
        perror("Output file open error");
        exit(EXIT_FAILURE);
    }
    dup2(out_fd, STDOUT_FILENO);
    close(out_fd);
}

// Process Stderr Redirection or Merging
if (merge_stderr) {
    // Merge stderr into stdout (2>&1)
    dup2(STDOUT_FILENO, STDERR_FILENO);
} else if (stderr_file && strlen(stderr_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (append_err ? O_APPEND : O_TRUNC);
    int err_fd = open(stderr_file, flags, 0644);
    if (err_fd < 0) {
        perror("Stderr file open error");
        exit(EXIT_FAILURE);
    }
    dup2(err_fd, STDERR_FILENO);
    close(err_fd);
}
}

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Pg      ^K Cut Text
^X Exit          ^J Justify       ^W Where is       ^V Next Pg      ^U UnCut Tex
```

~ - -zsh...memory\_and\_test\_shell.c~ - - nano signal\_handler.c~ - -zsh~ - - appen

UW PICO 5.09File: combined\_complex.c

```
        int arg_count = 0;
        char *arg_tok = strtok(cmds[i], " \t");
        while (arg_tok != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = arg_tok;
            arg_tok = strtok(NULL, " \t");
        }
        args[arg_count] = NULL;

        if (arg_count == 0) exit(EXIT_SUCCESS);

        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    }
}

// Parent closes all pipe descriptors
for (int i = 0; i < 2 * num_pipes; i++) {
    close(pipefds[i]);
}

// Wait for all child processes to exit
for (int i = 0; i < num_cmds; i++) {
    wait(NULL);
}
}

int main() {
    char input_buf[BUF_SIZE];
    while (1) {
        printf("\nComplex Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;
        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;
        execute_complex_pipeline(input_buf);
    }
    return 0;
}

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Pg      ^K Cut Text
^X Exit          ^J Justify       ^W Where is       ^V Next Pg      ^U UnCut Tex
```

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano combined_complex.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall combined_complex.c -o combined_complex
./combined_complex
```

```
Complex Shell > ls -la /invalid_folder /tmp &> combined.log
```

```
Complex Shell > ls -la /invalid_folder /tmp &> combined.log
```

```
Complex Shell > grep root < /etc/passwd | cut -d: -f1 > output_users.txt[cite: 7]
```

```
Complex Shell > exit
```

```
medikondarakesh@Medikondas-MacBook-Air ~ % █
```